

Databases

数据库完整学习讲义

清华大学统计与数据科学系 俞声 2026.5.27

第 1 章 数据库系统概述

1.1 什么是数据库系统

一个**数据库 (Database)** 是有组织的结构化信息集合，加上管理软件——**数据库管理系统 (DBMS)** ——共同实现数据的高效、安全、并发存取。

日常类比：电子表格 vs 数据库

- 电子表格（简单）：适合单用户、单张表、小数据
- 数据库（强大）：支持多用户、多关联表、大数据、复杂查询与崩溃恢复

1.2 DBMS 的五大核心能力

能力	说明
持久存储	数据写入磁盘，断电不丢失
高效检索	索引加速查询，百亿行毫秒级响应
并发控制	多用户同时读写，保证一致性
安全与访问控制	权限管理，防止未授权操作
数据完整性	约束规则自动拒绝无效数据

1.3 常见数据库类型

类型	数据格式	适用场景	典型产品
关系型	带严格 Schema 的表格	复杂查询 & ACID 事务	MySQL, PostgreSQL, SQLiteSQL,Server, Oracle
列式	稀疏宽列表	海量数据高吞吐扫描（时序/IoT）	Cassandra, HBase, Google Bigtable
文档型	JSON 文档，半结构化	快速迭代，灵活存储	DynamoDB,MongoDB, Couchbase, Amazon DynamoDB
图数据库	节点和边，深度关联	路径查找，关系分析	Neo4j, Amazon Neptune,Neo4j, ArangoDB
向量数据库	语义向量嵌入	AI/LLM 嵌入，语义搜索，RAG	FAISS, Pinecone, Milvus, pgvector

本讲义重点讲解使用最广泛的**关系型数据库**，非关系型数据库统称 NoSQL(not only sql)。

第 2 章 关系型数据库 vs DataFrame

2.1 表面相似，本质不同

关系型数据库 和 **DataFrame** 都以"行列表格"表示数据，都支持过滤、连接、聚合等操作，因此 SQL 初学时会觉得似曾相识。但本质大相径庭。

维度	DataFrame (Pandas)	关系型数据库
设计目的	单用户内存分析	支撑真实系统（电商/银行/企业应用）
数据规模	受内存限制	可存储海量持久数据
并发	无并发考量	多用户同时读写
数据约束	无强制规则	主键、外键、非空等约束自动执行
崩溃恢复	无	事务日志保证数据不丢失
思维方式	命令式：一步步操作	声明式：描述"想要什么"

2.2 声明式思维（Declarative Thinking）

SQL 是一种**声明式语言**：你只需描述**"想要什么结果"**，不需要指定**"如何计算"**。数据库引擎自动决定最优执行计划。

与 Pandas 的思维对比

- Pandas（命令式）：显式指定每一步 → `df.groupby(...).agg(...).sort_values(...)`
- SQL（声明式）：描述期望结果 → `SELECT ... FROM ... GROUP BY ... ORDER BY ...`
- 好处：同一条 SQL，换更快的执行计划，结果不变；Pandas 改写才能优化

2.3 规范化设计 (Normal Forms)

关系型数据库通常按**范式 (Normal Forms)** 设计，要求不同类型的信息存储在不同的表中，避免冗余和不一致。

范式	规则	解决的问题
1NF（第一范式）	每个单元格只存一个原子值，不允许列表/数组	消除重复组
2NF（第二范式）	去除对联合主键部分列的依赖，拆分成独立表	消除部分函数依赖
3NF（第三范式）	非键列必须直接依赖主键，不能依赖其他非键列	消除传递依赖

实践中最常用到 **3NF**。正因范式要求，数据被分散存储在多张表中，简单的业务问题也往往需要跨表查询。

第 3 章 Chinook 数据集与 Schema 设计

3.1 Chinook 业务场景

Chinook 是 SQLite 官方示例数据库，模拟一家**数字音乐商店**。顾客可以购买单曲，曲目属于专辑，专辑属于艺术家，购买记录以发票形式存储。

典型购买流程

- 1. 顾客下单
- 2. 系统生成一张发票（Invoice）
- 3. 发票中包含多个购买条目（InvoiceLine），每条对应一首曲目

3.2 核心表与关系链

表名	含义	主键
Customer	购买者信息	CustomerId
Invoice	交易记录（发票头）	InvoiceId
InvoiceLine	发票明细（购买的具体曲目）	InvoiceLineId
Track	曲目（商品）	TrackId
Album	专辑（曲目集合）	AlbumId
Artist	艺术家（创作者）	ArtistId

关系链：**Customer → Invoice → InvoiceLine → Track → Album → Artist**

即便是「某顾客买了哪首歌」这样的简单查询，也需要跨越这 4 张表！这正是范式设计的结果。

3.3 为什么不用一张大表？

如果把所有信息合并到一张表，会出现：

大表的问题

- 数据冗余：艺术家名字在每首曲目行中重复存储
- 更新异常：修改艺术家名字需要更新所有相关行，容易漏改
- 插入异常：新艺术家没有专辑时无法插入
- 删除异常：删除最后一张专辑时，艺术家信息也丢失

3.4 SQLite 简介

SQLite 是轻量级"无服务器"DBMS，无需安装配置，数据库就是一个本地文件（.sqlite 或 .db），非常适合学习和小型应用（手机 App、桌面软件、嵌入式设备）。

特性	说明
无服务器	不需要单独启动数据库服务进程
零配置	下载即用，无需用户管理
单文件	整个数据库存储在一个 .sqlite/.db 文件中
单用户	无并发控制，适合学习和轻量应用

连接命令：

```
sqlite3 Chinook_Sqlite.sqlite
```

第 4 章 Schema: 表结构设计

4.1 什么是 Schema

Schema 定义了数据库的结构，包括有哪些表、每张表有哪些列、列的数据类型，以及表之间的关系（约束）。Schema 是数据的"蓝图"。

SQLite 中查看 Schema 的命令

```
.tables      ← 列出数据库中所有表
.schema      ← 列出所有表的建表语句
.schema Album ← 查看 Album 表的建表语句
```

4.2 CREATE TABLE 语法

CREATE TABLE 是定义表结构的 SQL 语句。即使不自己设计数据库，也必须看懂它，才能理解字段类型、约束和表间关系，从而写出正确的 JOIN。

以 Album 表为例：

```
CREATE TABLE Album
(
    AlbumId INTEGER NOT NULL,    -- 专辑 ID (整数，不为空)
    Title   NVARCHAR(160) NOT NULL, -- 专辑标题 (字符串，最长 160)
    ArtistId INTEGER NOT NULL,    -- 艺术家 ID (外键)

    CONSTRAINT PK_Album PRIMARY KEY (AlbumId),    -- 主键约束
    FOREIGN KEY (ArtistId) REFERENCES Artist (ArtistId) -- 外键
        ON DELETE NO ACTION ON UPDATE NO ACTION
);
CREATE INDEX IFK_AlbumArtistId ON Album (ArtistId); -- 索引
```

4.3 主键 (Primary Key)

主键 是能唯一标识一行的列（或列组合）。主键自动具备 NOT NULL 和 UNIQUE 属性，是其他表通过外键引用本表时的依据。

主键的作用

- 唯一标识：保证每一行都能被精确找到

- 被引用的锚点：其他表的外键指向本表主键
- 自动索引：主键列自动建立索引，查询极快

4.4 外键 (Foreign Key)

外键 是本表中引用另一张表主键的列，用于建立表间关系，并保证**引用完整性**。

Album.ArtistId → Artist.ArtistId：意味着每张专辑必须关联一个真实存在的艺术家。

ON DELETE / ON UPDATE 行为

NO ACTION ← 若有专辑引用该艺术家，则禁止删除/修改艺术家 ID

CASCADE ← 删除艺术家时，同时删除其所有专辑（慎用！）

SET NULL ← 删除艺术家时，将专辑的 ArtistId 置为 NULL

4.5 索引 (Index)

索引帮助数据库快速定位数据，无需逐行扫描。本质是"以空间换时间"。

特性	说明
加速读取	大幅提升 WHERE、JOIN、ORDER BY、GROUP BY 性能
自动维护	每次 INSERT/UPDATE/DELETE 时同步更新，写入有额外开销
主键/唯一列	自动建立索引
外键列	不自动建立，需手动 CREATE INDEX

```
CREATE INDEX IFK_AlbumArtistId ON Album (ArtistId);
```

索引过多会拖慢写入性能，应重点对 WHERE / JOIN / ORDER BY 频繁使用的列建索引。

4.6 数据操作简介 (INSERT / UPDATE / DELETE)

数据库同样支持写入操作，但有约束保护：

```
-- INSERT: 插入两行专辑记录
INSERT INTO Album (AlbumId, Title, ArtistId) VALUES
    (1, 'For Those About To Rock We Salute You', 1),
```

```
(5, 'Big Ones', 3);
```

```
-- 若 ArtistId=999 不存在于 Artist 表中，数据库会拒绝插入！
```

SQL 字符串引号规则

- 单引号 ' '：用于字符串值，如 'USA'
- 双引号 " "：用于含空格的表名/列名，如 "My Table"
- SQLite 额外支持 [] (SQL Server 风格) 和 `` (MySQL 风格)

第 5 章 查询数据库：SELECT 语句

5.1 SQL 查询思维模型

每条 SQL 查询都回答一个具体问题：

子句	作用	类比
FROM	指定数据来源（哪张/哪些表）	df = pd.read_csv(...)
WHERE	过滤行（满足条件的行才保留）	df[df.condition]
GROUP BY	按列分组	df.groupby(...)
HAVING	过滤分组（聚合后的条件）	.filter(...)（分组后）
SELECT	选取列或计算表达式	df[[cols]]
ORDER BY	排序	.sort_values(...)
LIMIT / OFFSET	限制返回行数 / 分页	.head(n) / 切片

SQL 执行顺序 ≠ 书写顺序

书写顺序：SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY → LIMIT
执行顺序：FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT
理解执行顺序有助于避免常见错误（如 WHERE 中无法引用 SELECT 的别名）

5.2 基础语法与注意事项

```
-- 查询所有顾客的姓名
SELECT FirstName, LastName
FROM Customer;

-- SELECT * 表示选取所有列（生产环境慎用）
SELECT * FROM Customer;
```

SQL 语法规则

- 关键字大小写不敏感：SELECT = select = Select

- 列名大小写不敏感: FirstName = firstname
- 字符串值大小写敏感! "USA" ≠ "usa"
- 换行/缩进不影响执行, 只为可读性
- 语句必须以分号; 结尾 (多语句环境)

5.3 LIMIT + OFFSET (分页)

生产数据库可能有数十亿行, 不加限制地 SELECT 会崩溃客户端, 甚至拖垮服务器。

```
-- 只取前 10 行
SELECT FirstName, LastName FROM Customer LIMIT 10;

-- 分页: 跳过前 10 行, 取第 11-20 行
SELECT FirstName, LastName FROM Customer LIMIT 10 OFFSET 10;

-- 更高效的"键集分页" (适合大表)
SELECT FirstName, LastName, CustomerId
FROM Customer
WHERE CustomerId > 10    -- 从上次取到的最后一个 ID 开始
ORDER BY CustomerId
LIMIT 10;
```

键集分页原理: OFFSET 越大越慢 (需扫描跳过的行); 键集分页利用索引直接定位, 始终 O(1)。

5.4 WHERE 条件过滤

```
-- 等值过滤
SELECT * FROM Customer WHERE Country = 'USA';

-- 不等
SELECT * FROM Customer WHERE Country != 'USA';

-- 多条件 (AND / OR)
SELECT * FROM Customer WHERE Country = 'USA' OR Country = 'Canada';

-- 范围列表
SELECT * FROM Customer WHERE Country IN ('USA', 'Canada', 'Brazil');

-- 模糊匹配 (LIKE)
SELECT * FROM Customer WHERE Phone LIKE '+1 %'; -- % 匹配任意多字符
-- _ 匹配单个字符; LIKE 大小写敏感; 仅 '前缀%' 模式能用到索引
```

5.5 随机抽样 (Sampling)

```
-- 随机取 10 行 (简单随机抽样)
SELECT FirstName, LastName FROM Customer
ORDER BY RANDOM()
LIMIT 10;

-- 注意: LIMIT 不是随机样本, ORDER BY RANDOM() 才是
-- 大表上速度较慢; 专业 DBMS 有更高效率的采样函数
```

5.6 聚合函数 (Aggregation)

常用聚合函数: **COUNT / SUM / AVG / MIN / MAX**

```
-- 总顾客数
SELECT COUNT(*) FROM Customer;

-- 每个国家的顾客数 (降序)
SELECT Country, COUNT(*)
FROM Customer
GROUP BY Country
ORDER BY COUNT(*) DESC;

-- 只保留顾客数 >= 5 的国家 (HAVING 过滤分组结果)
SELECT Country, COUNT(*)
FROM Customer
GROUP BY Country
HAVING COUNT(*) >= 5;
```

WHERE vs HAVING

WHERE → 在分组前过滤行 (作用于原始数据)

HAVING → 在分组后过滤组 (作用于聚合结果)

语法顺序: WHERE ... GROUP BY ... HAVING ...

5.7 子查询 (Subquery) 与 EXISTS

```
-- 查询有购买记录的顾客 (EXISTS 写法)
SELECT FirstName, LastName FROM Customer c
WHERE EXISTS (
    SELECT 1 FROM Invoice i
    WHERE i.CustomerId = c.CustomerId
);

-- 等价的 IN 写法
```

```
SELECT FirstName, LastName FROM Customer c
WHERE c.CustomerId IN (
    SELECT DISTINCT CustomerId FROM Invoice
);
```

-- 消费超过平均水平的顾客（嵌套子查询）

```
SELECT c.FirstName, c.LastName, SUM(i.Total) AS total_spent
FROM Customer c
JOIN Invoice i ON c.CustomerId = i.CustomerId
GROUP BY c.CustomerId
HAVING SUM(i.Total) > (
    SELECT AVG(customer_total)
    FROM (
        SELECT SUM(Total) AS customer_total
        FROM Invoice
        GROUP BY CustomerId
    )
);
```

第 6 章 多表联接 (JOIN)

6.1 JOIN 类型概览

类型	含义	结果
INNER JOIN (默认 JOIN)	只保留两表都匹配的行	去掉任意一侧无对应记录的行
LEFT JOIN	保留左表所有行，右表无匹配则填 NULL	左表完整，右表可为空
RIGHT JOIN	保留右表所有行 (SQLite 不支持)	右表完整，左表可为空
FULL OUTER JOIN	两侧都保留 (SQLite 不支持)	两侧都完整，可互有 NULL

6.2 基础 INNER JOIN

查询每位顾客的总消费金额：

```
SELECT concat(c.FirstName, ' ', c.LastName) AS Name,
       SUM(i.Total) AS Total
FROM Customer c           -- 表别名 c
JOIN Invoice i             -- 表别名 i
    ON c.CustomerId = i.CustomerId
GROUP BY i.CustomerId;

-- AS 关键字可省略于表别名（但列别名最好保留 AS，更清晰）
```

6.3 LEFT JOIN

LEFT JOIN 保留左表的所有记录，即使右表中没有匹配。适合"包含零值"的统计。

```
-- 查询每首曲目的销量（包括从未卖出的曲目，销量显示为 0）
SELECT t.Name AS Song, COUNT(il.InvoiceLineId) AS Sales
FROM Track t
LEFT JOIN InvoiceLine il ON il.TrackId = t.TrackId
GROUP BY t.TrackId
ORDER BY Sales DESC;

-- 若改为 INNER JOIN，则未售出曲目不会出现在结果中
```

6.4 多表 JOIN

查询每位顾客购买了哪首歌，需跨越 4 张表：

```
SELECT t.Name AS Song, c.LastName
FROM InvoiceLine il
JOIN Track t      ON il.TrackId  = t.TrackId
JOIN Invoice i    ON il.InvoiceId = i.InvoiceId
JOIN Customer c  ON i.CustomerId = c.CustomerId;
```

多表 JOIN 技巧：以"起点表"为 FROM，沿外键关系链逐步 JOIN，每次只添加下一张需要的表。

6.5 六表 JOIN 综合案例

每位顾客购买了每位艺术家多少首歌？

```
SELECT c.CustomerId,
       concat(c.FirstName, ' ', c.LastName) AS customer_name,
       ar.Name AS artist_name,
       COUNT(*) AS num_tracks_purchased
FROM Customer c
JOIN Invoice      i  ON c.CustomerId = i.CustomerId
JOIN InvoiceLine il ON i.InvoiceId  = il.InvoiceId
JOIN Track       t  ON il.TrackId   = t.TrackId
JOIN Album       al ON t.AlbumId    = al.AlbumId
JOIN Artist      ar ON al.ArtistId  = ar.ArtistId
GROUP BY c.CustomerId, ar.ArtistId
ORDER BY c.CustomerId, num_tracks_purchased DESC;
```

关系链回顾

Customer → Invoice → InvoiceLine → Track → Album → Artist

每个箭头对应一个 JOIN ON 外键 = 主键

第 7 章 在 Python 中使用 SQLite

7.1 为什么在 Python 中使用 SQL

真实的数据科学工作流中，数据通常存储在数据库里，而非 CSV。Python 的角色是：

典型工作流

1. 用 SQL 在数据库中做数据提取和聚合（高效，数据库优化器负责）
2. 将结果导入 Pandas DataFrame
3. 用 Python 进行进一步分析、可视化或机器学习建模

7.2 连接数据库

Python 内置 `sqlite3` 包，无需额外安装：

```
import sqlite3

# 连接（如果文件不存在则自动创建）
conn = sqlite3.connect("Chinook.sqlite")

# 操作完成后关闭连接
conn.close()
```

7.3 用 Pandas 执行 SQL 查询

最推荐的方式：`pd.read_sql()` 直接返回 DataFrame。

```
import sqlite3
import pandas as pd

conn = sqlite3.connect("Chinook.sqlite")

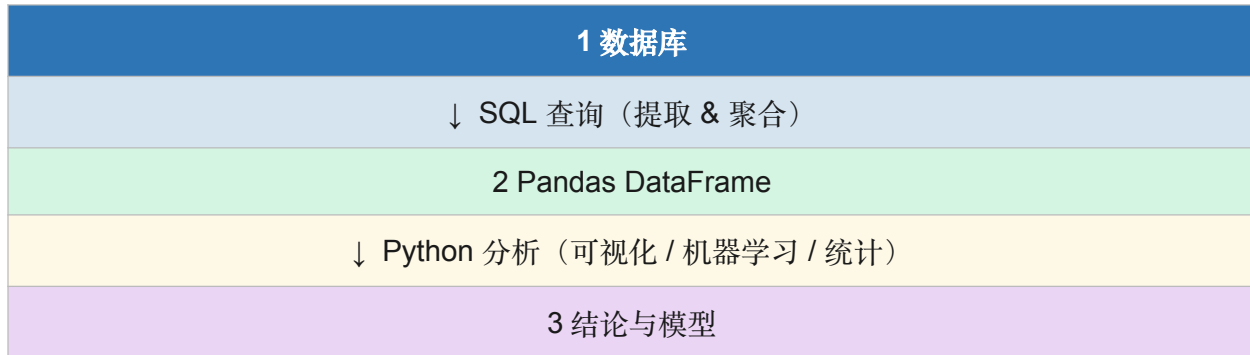
query = """
    SELECT BillingCountry AS country,
           SUM(Total)      AS revenue
    FROM Invoice
    GROUP BY country
    ORDER BY revenue DESC
    """

df = pd.read_sql(query, conn)
```

```
print(df.head())
```

```
# 后续可用 Pandas / matplotlib / sklearn 继续分析
```

7.4 完整工作流示意



第 8 章 SQL 速查卡

8.1 SELECT 完整语法模板

```
SELECT [DISTINCT] 列 1, 列 2, 聚合函数(列) AS 别名
FROM 表 1 [别名]
[JOIN 表 2 [别名] ON 联接条件]
[WHERE 行过滤条件]
[GROUP BY 分组列]
[HAVING 分组过滤条件]
[ORDER BY 列 [ASC|DESC]]
[LIMIT n [OFFSET m]];
```

8.2 常用运算符速查

运算符	说明	示例
=	等于	Country = 'USA'
!= 或 <>	不等于	Country != 'USA'
> >= < <=	数值比较	Total > 10
AND / OR / NOT	逻辑组合	Country = 'USA' OR Country = 'Canada'
IN (...)	成员判断	Country IN ('USA', 'Canada')
BETWEEN a AND b	范围（含端点）	Total BETWEEN 5 AND 20
LIKE ...	模糊匹配（% 任意，_ 单字符）	Phone LIKE '+1 %'
IS NULL / IS NOT NULL	空值判断	Company IS NULL

8.3 聚合函数速查

函数	说明
COUNT(*)	统计所有行数（包含 NULL）
COUNT(列)	统计非 NULL 的行数

COUNT(DISTINCT 列)	统计去重后的非 NULL 值数
SUM(列)	求和
AVG(列)	平均值（自动忽略 NULL）
MIN(列) / MAX(列)	最小值 / 最大值

8.4 常见错误汇总

新手常见错误

1. WHERE 中使用聚合函数 → 应改用 HAVING
2. 字符串用双引号 → 应用单引号 'USA'（双引号是标识符引用符）
3. LIKE 大小写敏感，'usa' 匹配不到 'USA'
4. INNER JOIN 丢失零值行 → 需要 LEFT JOIN
5. GROUP BY 分组列与 SELECT 不一致 → 报错或行为未定义
6. ORDER BY 在 WHERE/GROUP BY 之前书写 → 语法错误
7. LIMIT 取的是插入顺序前 N 行，不是随机样本